

Spring Framework

Autowiring in Spring :-

What is Autowiring ?

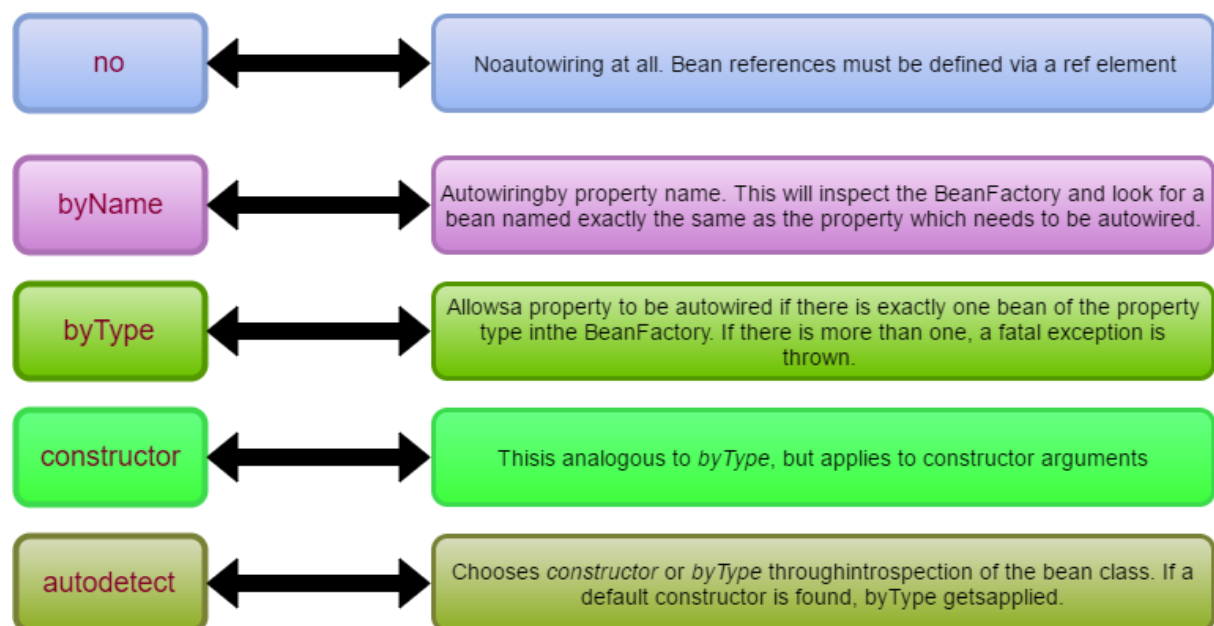
- Autowiring in Spring automatically injects required dependencies into a bean.
- It reduces manual configuration in the application.
- Spring searches and provides matching beans based on type, name, or constructor.
- It simplifies dependency injection and improves maintainability.

Why we use Autowiring ?

- Automatically injects required dependencies.
- Reduces manual XML or annotation configuration.
- Minimizes boilerplate setter and constructor code.
- Lets Spring detect and match beans automatically.

Modes of Autowiring :-

Here's a breakdown of the primary modes of autowiring available in Spring Boot:



1. no

- No autowiring is used.
- All bean references must be defined manually using a ref element.

Example -

```
<bean id="b" class="Demo.B"></bean>
<bean id="a" class="Demo.A" autowire="no"></bean>
```

Output –

```
A class constructor
I am display method - B Class
```

2. byName

- Autowiring is done by matching the property name.
- Spring looks for a bean whose name matches the property name.

Example –

```
<bean id="b" class="Demo.B"></bean>
<bean id="a" class="Demo.A" autowire="byName"></bean>
```

Output –

```
A class constructor
B class constructor
I am display method - B Class
I am print method - B Class
```

3. byType

- Spring autowires a property if exactly one bean of the required type exists.
- If multiple beans of the same type are found, Spring throws an error.

Example –

```
<bean id="b1" class="Demo.B"></bean>
<bean id="a" class="Demo.A" autowire="byType"></bean>
```

Output –

```
A class constructor
B class constructor
I am display method - B Class
I am print method - B Class
```

4. constructor

- Similar to byType, but applied to constructor arguments.
- Spring matches constructor parameter types with beans in the container.

Example –

```
<bean id="b" class="Demo.B"></bean>
<bean id="a" class="Demo.A" autowire="constructor"></bean>
```

Output –

```
A class constructor
I am display method - B Class
```

5. autodetect (Deprecated)

- Spring first tries constructor autowiring.
- If a default constructor is found, it switches to byType.

Implementation of Autowiring :-

A.java –

```
package Demo;

public class A {

    B b; //Autowiring

    public B getB() {
        return b;
    }

    public void setB(B b) {
        this.b = b;
    }

    A(){
        System.out.println("A class constructor");
    }

    void display() {
        System.out.println("I am display method - B Class");
        b.print();
    }
}
```

B.java –

```
package Demo;

public class B {
    B(){
        System.out.println("B class constructor");
    }

    void print() {
        System.out.println("I am print method - B Class");
    }
}
```

Test.java –

```
package Demo;

import org.springframework.beans.factory.BeanFactory;
import org.springframework.beans.factory.xml.XmlBeanFactory;
import org.springframework.core.io.ClassPathResource;

public class Test {

    public static void main(String[] args) {
        BeanFactory fact=new XmlBeanFactory(new ClassPathResource("applicationContext.xml"));
        A a1=fact.getBean(A.class);
        a1.display();
    }

}
```

applicationContext.xml –

```
https://www.springframework.org/schema/beans/spring-beans.xsd (xsi:schemaLocation)
<?xml version="1.0" encoding="UTF-8"?>
<beans
    xmlns="http://www.springframework.org/schema/beans"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xmlns:p="http://www.springframework.org/schema/p"
    xsi:schemaLocation="http://www.springframework.org/schema/beans
                        https://www.springframework.org/schema/beans/spring-beans.xsd">

    <!-- <bean id="b" class="Demo.B"></bean>
    <bean id="a" class="Demo.A" autowire="byName"></bean> -->

    <!-- <bean id="b" class="Demo.B"></bean>
    <bean id="a" class="Demo.A" autowire="constructor"></bean> -->

    <!--<bean id="b" class="Demo.B"></bean>
    <bean id="a" class="Demo.A" autowire="no"></bean>-->

    <!--<bean id="b1" class="Demo.B"></bean>
    <bean id="a" class="Demo.A" autowire="byType"></bean> -->

</beans>
```

Advantages of Autowiring

1. **Reduces boilerplate code** – No need to manually write setters or constructor injections.
2. **Faster development** – Spring automatically handles dependency injection.
3. **Improves readability** – Configuration becomes cleaner and easier to understand.
4. **Less XML configuration** – Annotations minimize external config files.
5. **Loose coupling** – Dependencies are injected dynamically, improving flexibility.

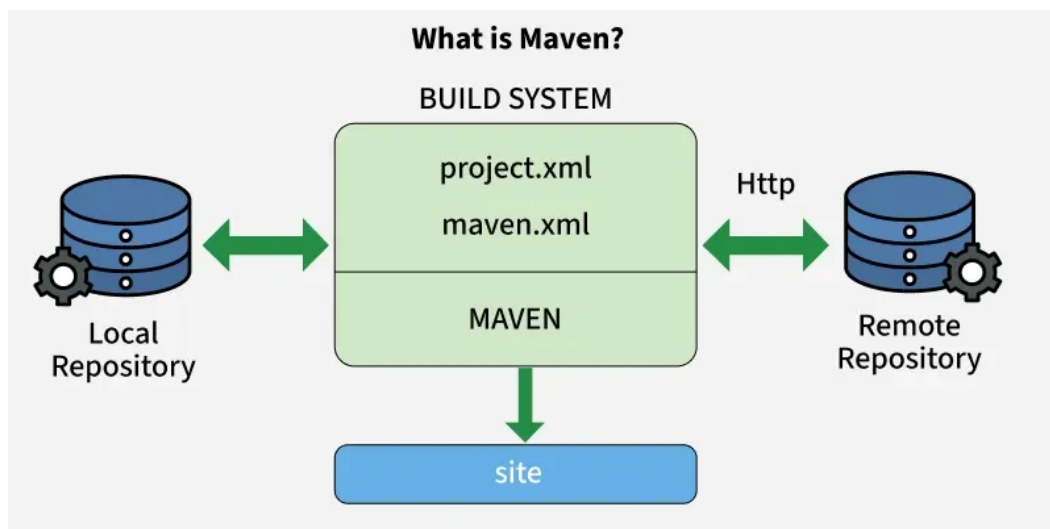
Disadvantages of Autowiring

1. **Less control** – Spring decides which bean to inject, which may cause confusion.
2. **Debugging becomes harder** – Automatic wiring makes issues difficult to trace.
3. **Multiple bean conflicts** – If more than one matching bean exists, Spring may throw errors.
4. **Implicit behavior** – Developers may not realize which bean is being injected.
5. **Not suitable for large complex applications** – Manual wiring is sometimes safer for clarity.

Maven :-

What is Maven ?

- Maven is a build automation and project management tool for Java.
- It manages project dependencies automatically.
- It uses a file called pom.xml to configure the project.
- It simplifies compiling, building, and running Java applications.



When to use Maven ?

1. To manage project dependencies automatically.
2. To maintain a standard and consistent project structure.
3. To automate build tasks like compile, test, and package.
4. To ensure consistent builds across different environments.
5. To handle multi-module projects efficiently.
6. To simplify project setup using archetypes.

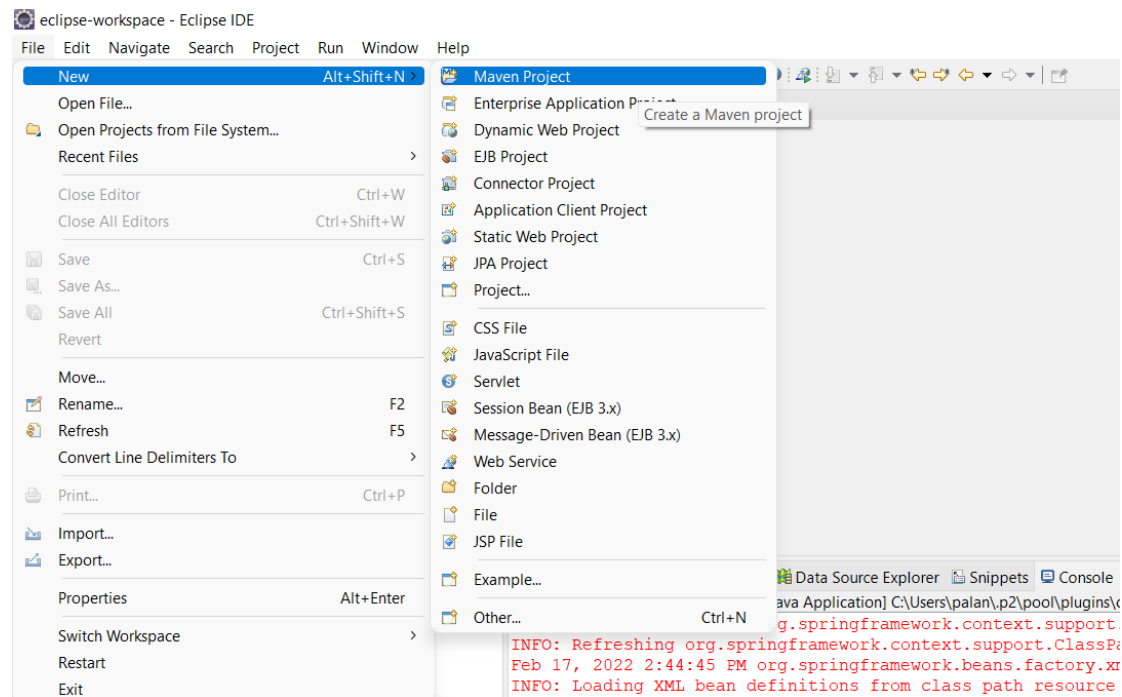
Features of Maven :-

1. **Project Management:** Provides a standard project structure and easy lifecycle management.
2. **Dependency Management:** Automatically downloads and manages libraries with transitive dependency support.
3. **Build Automation:** Handles compilation, packaging, testing, and deployment with a single command.
4. **Plugin Support:** Offers a rich plugin ecosystem to extend build functionality.
5. **Consistent Builds:** Ensures repeatable builds across different environments.
6. **Documentation Generation:** Can automatically generate project reports and documentation.

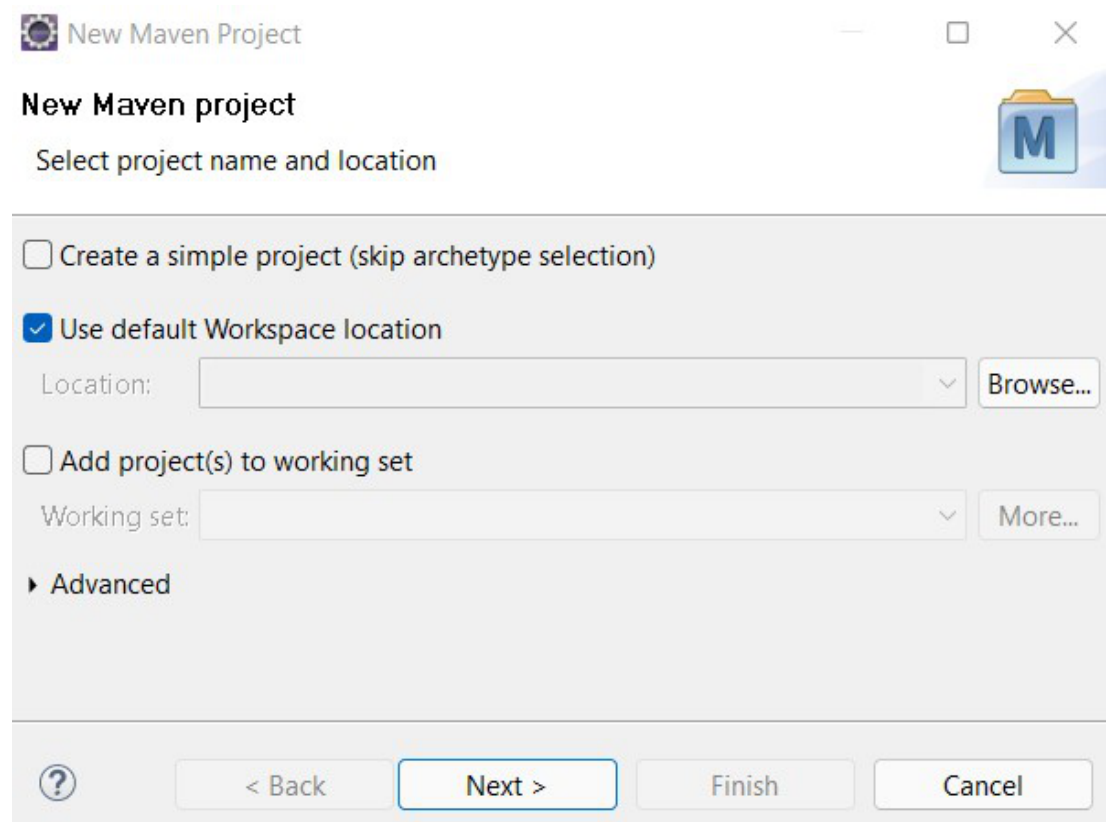
How to Create a Maven Project in Eclipse IDE?

Step 1: Create A New Maven Project

First, we will create a new Maven project. For this, we will open our Eclipse IDE, then go to File > New > Maven Project.



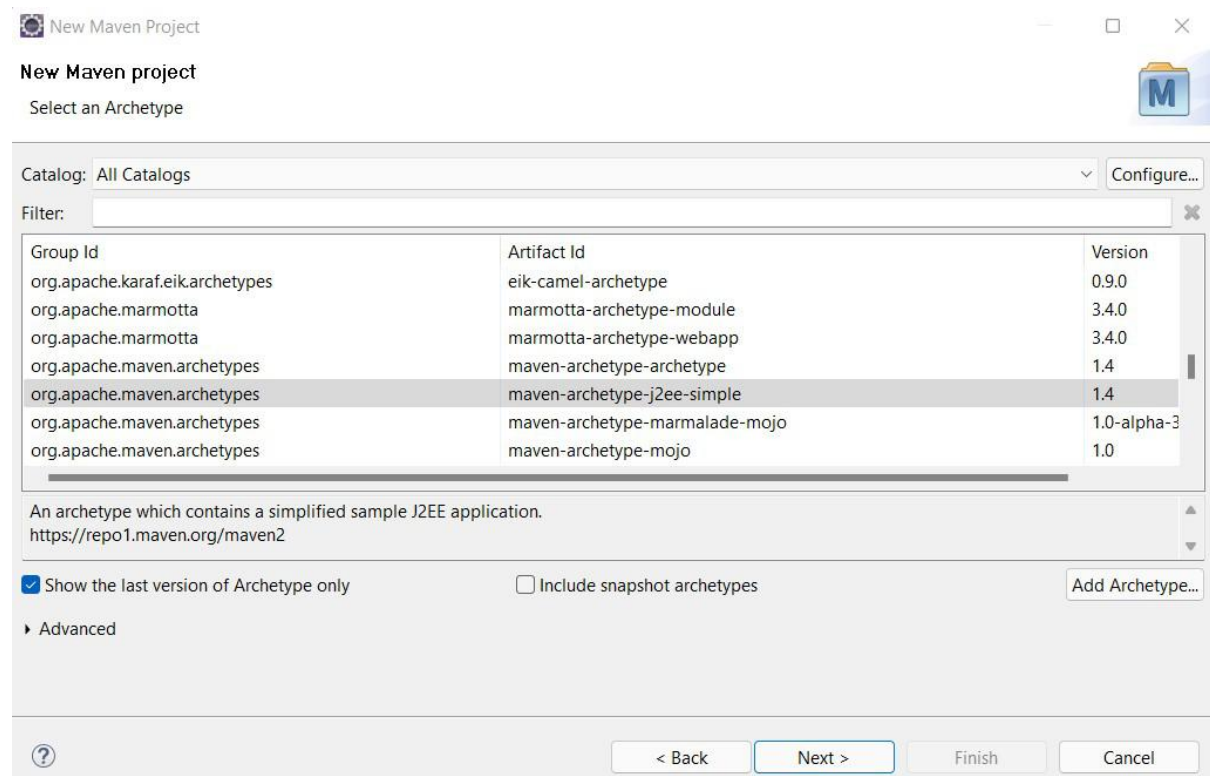
Once we click on Maven Project we will get a prompt for setting up our Workspace location.



After setting Workspace location according to our choice. We will click on Next to go to further project setup steps.

Step 2: Setting Up Archetype

We select a Maven archetype, which is a template used to generate a basic project structure. Here, we use the maven-archetype-j2ee-simple archetype to quickly create a starter J2EE project.



Archetype Selection

After choosing the archetype we will click on Next and it will ask for the details like Group Id, Artifact Id, and Package information.

- Group Id - It's a unique identifier to uniquely identify the project in a maven repository.
- Artifact Id - It's basically the project name.

We will be using the following details:

New Maven Project

Specify Archetype parameters

Group Id:

Artifact Id:

Version:

Package:

Properties available from archetype:

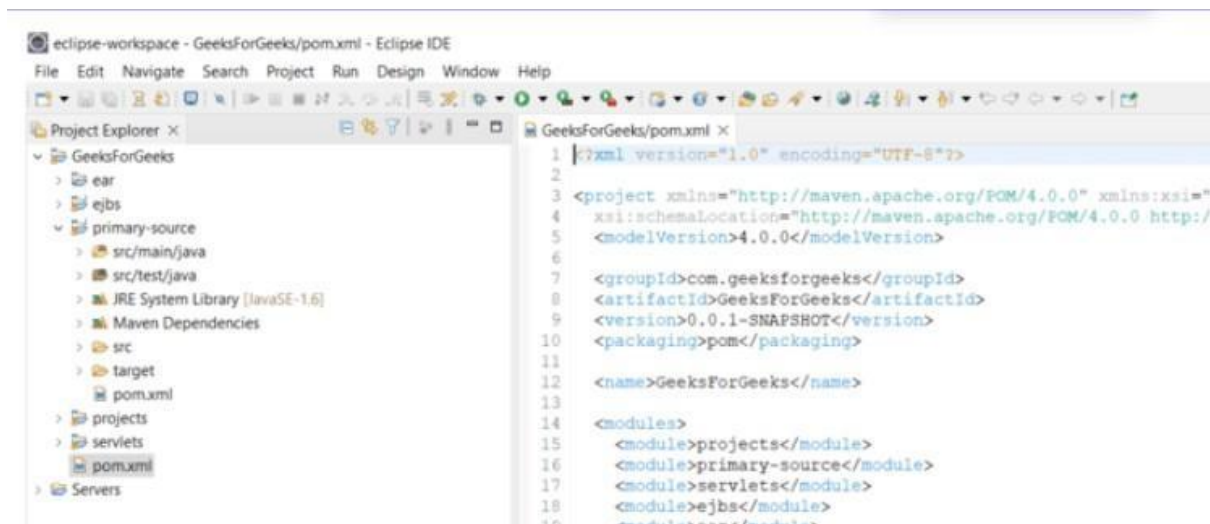
Name	Value

Advanced

< Back Next > Finish Cancel

Maven Project Details

After adding all these details we need to click on the Finish button. Maven will start creating your project and after a few seconds, your project will be created.



Maven project structure

By creating maven project we will use maven's powerful features, you can automate tasks, manage project dependencies efficiently, and integrate with other tools like Jenkins for continuous integration.